Optimización en Ciencia de Datos

Clase 4: métodos de descenso para aprendizaje automático

Pablo A. Lotito Victoria Orlando

Modelización Matemática
Facultad de Ciencias Exactas – UNCPBA

2.º cuatrimestre 2026









Tabla de Contenidos

1. Métodos de búsqueda lineal
2. Métodos de primer orden en $\mathbb{R}^n$
3. Métodos de descenso en ML
   - Momentum y Nesterov
4. Resumen
5. Para leer

Puente con lo que ya vimos en Modelización

Ya lo vimos en la U4 (clases 5, 6 y 8)

- Dirección de descenso y esquema general.
- Gradiente con paso fijo, exacto e inexacto (Armijo, Wolfe).
- Newton y cuasi-Newton; gradiente conjugado.
- Convergencia en cuadráticas y número de condición κ.
- Notebooks de funciones cuadráticas y no cuadráticas.

Lo nuevo de hoy

- Por qué el gradiente completo no escala cuando n es grande.
- Descenso estocástico (SGD) y mini-batch.
- Momentum y Nesterov.
- Métodos adaptativos: AdaGrad, RMSProp, Adam y AdamW.
- Calibración del paso y validación.

Las diapositivas marcadas con Visto en U4 son repaso: se pueden recorrer rápido.

Métodos numéricos de optimización

Métodos iterativos

$$x_{n+1} = F_n(x_n)$$

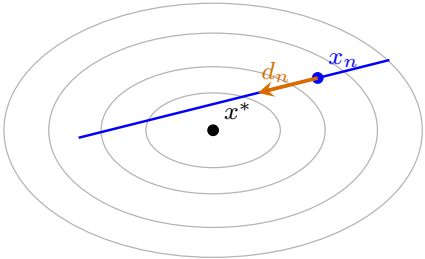
- Métodos de búsqueda lineal (en $\mathbb{R}$)
- Métodos de primer orden: GD, SGD Momentum, Nesterov.
- Métodos de segundo orden: Newton, Levenberg-Marquardt.

Métodos de búsqueda lineal

Minimización en una dimensión:

$$\phi(\alpha) = f(x_n + \alpha\, d_n)$$

$$\phi'(0) = \nabla f(x_n)^\top d_n$$

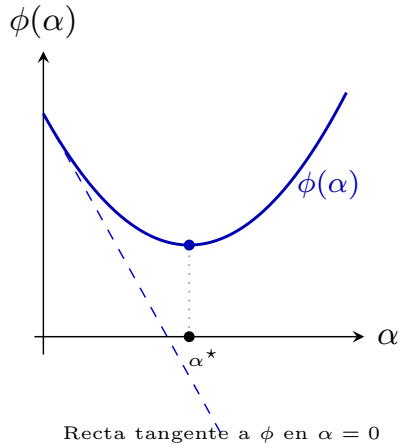


Métodos de búsqueda lineal

Minimización en una dimensión:

$$\phi(\alpha) = f(x_n + \alpha\, d_n)$$

$$\phi'(0) = \nabla f(x_n)^\top d_n$$



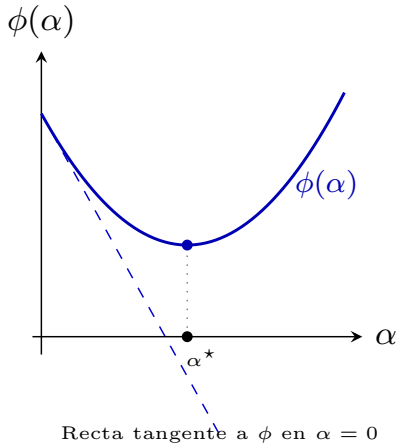
Métodos de búsqueda lineal

Minimización en una dimensión:

$$\phi(\alpha) = f(x_n + \alpha\, d_n)$$

$$\phi'(0) = \nabla f(x_n)^\top d_n$$

- Sin derivadas: Uniforme/dicotómico/Golden section/Fibonacci.
- Con derivadas: bisección, Newton
- Métodos inexactos: Armijo, Wolfe, Goldstein.

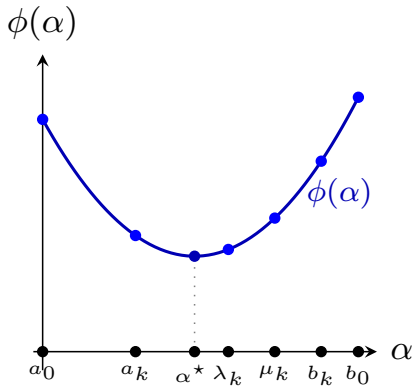


Métodos de búsqueda lineal sin derivadas

Si ϕ es cuasi-convexa, $a_0 = a$, $b_0 = b$, $\alpha^* \in [a_k, b_k]$ cada vez más pequeño.

Si $a_k \leq \lambda_k \leq \mu_k \leq b_k$

- $\phi(\lambda_k) \leq \phi(\mu_k) \rightarrow a_{k+1} = \lambda_k,\ b_{k+1} = b_k.$
- $\phi(\mu_k) \geq \phi(b_k) \rightarrow a_{k+1} = a_k,\ b_{k+1} = \mu_k.$

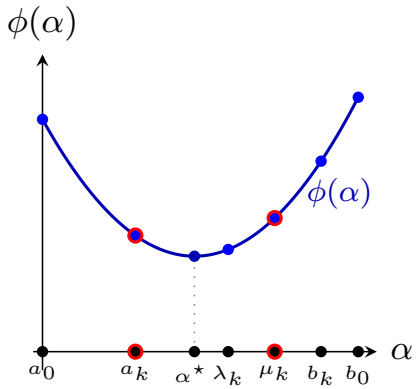


Métodos de búsqueda lineal sin derivadas

Si ϕ es cuasi-convexa, $a_0 = a$, $b_0 = b$, $\alpha^* \in [a_k, b_k]$ cada vez más pequeño.

Si $a_k \leq \lambda_k \leq \mu_k \leq b_k$

- $\phi(\lambda_k) \leq \phi(\mu_k) \rightarrow a_{k+1} = \lambda_k,\ b_{k+1} = b_k.$
- $\phi(\mu_k) \geq \phi(b_k) \rightarrow a_{k+1} = a_k,\ b_{k+1} = \mu_k.$



Métodos de búsqueda lineal sin derivadas

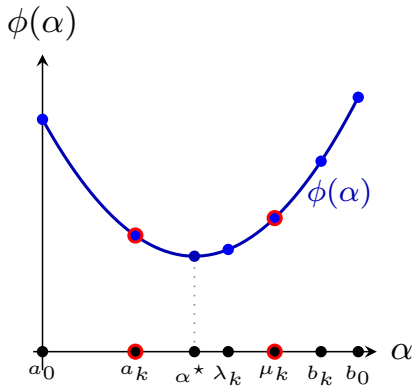
Si ϕ es cuasi-convexa, $a_0 = a$, $b_0 = b$, $\alpha^* \in [a_k, b_k]$ cada vez más pequeño.

Si $a_k \leq \lambda_k \leq \mu_k \leq b_k$

- $\phi(\lambda_k) \leq \phi(\mu_k) \rightarrow a_{k+1} = \lambda_k,\ b_{k+1} = b_k.$
- $\phi(\mu_k) \geq \phi(b_k) \rightarrow a_{k+1} = a_k,\ b_{k+1} = \mu_k.$

Según la definición de λ y μ trés métodos:

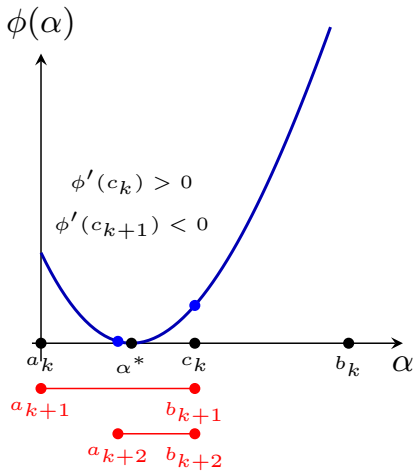
- **Dicotómico**: $\lambda_k, \mu_k = \frac{a_k + b_k}{2} \pm \epsilon$
- **Golden section**: $\mu_{k+1} = a_{k+1} + (b_{k+1} - a_{k+1})\frac{\sqrt{5}-1}{2}$
- **Fibonacci**: $\mu_{k+1} = a_{k+1} + (b_{k+1} - a_{k+1})\frac{F_{n-k-1}}{F_{n-k}}$



Métodos de búsqueda lineal con derivadas

Si ϕ es cuasi-convexa y derivable ($\phi'(\alpha^*) \neq 0$)

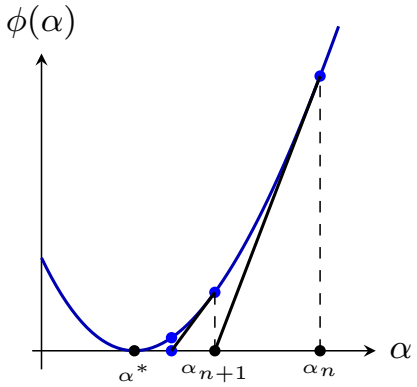
- **Bisección**: $c_k = \frac{a_k + b_k}{2}$, si $\phi'(c_k) > 0$ $a_{k+1} = c_k$



Métodos de búsqueda lineal con derivadas

Si ϕ es cuasi-convexa y derivable ($\phi'(\alpha^*) \neq 0$)

- **Bisección**: $c_k = \frac{a_k + b_k}{2}$, si $\phi'(c_k) > 0$ $a_{k+1} = c_k$
- **Newton**: $\alpha_{k+1} = \alpha_k - \frac{\phi'(\alpha_k)}{\phi''(\alpha_k)}$



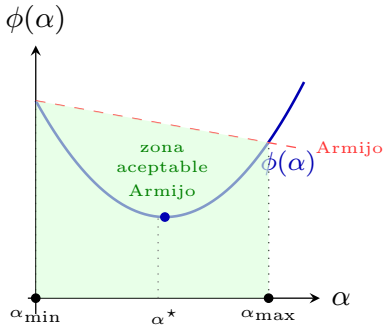
Métodos de búsqueda lineal inexactos (usan derivada)

No buscan el mínimo sino un descenso *suficiente*.

Métodos de búsqueda lineal inexactos (usan derivada)

No buscan el mínimo sino un descenso *suficiente*.
Armijo (condición suficiente de descenso):

$$\phi(\alpha) \leq \phi(0) + c_1\,\alpha\,\phi'(0), \quad c_1 \in (0,1)$$



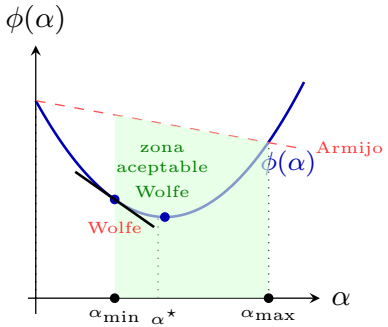
Métodos de búsqueda lineal inexactos (usan derivada)

No buscan el mínimo sino un descenso *suficiente*.
Armijo (condición suficiente de descenso):

$$\phi(\alpha) \le \phi(0) + c_1\,\alpha\,\phi'(0), \quad c_1 \in (0,1)$$

Wolfe (Armijo + curvatura):

$$\phi'(\alpha) \ge c_2\,\phi'(0), \quad 0 < c_1 < c_2 < 1$$



Métodos de búsqueda lineal inexactos (usan derivada)

No buscan el mínimo sino un descenso *suficiente*.

Armijo (condición suficiente de descenso):

$$\phi(\alpha) \leq \phi(0) + c_1\,\alpha\,\phi'(0), \quad c_1 \in (0,1)$$

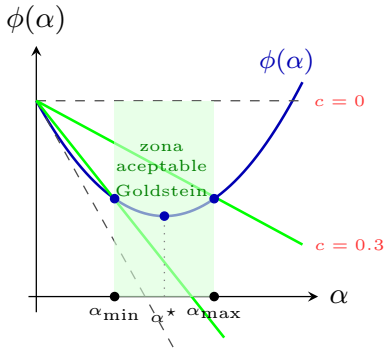
Wolfe (Armijo + curvatura):

$$\phi'(\alpha) \geq c_2\,\phi'(0), \quad 0 < c_1 < c_2 < 1$$

Goldstein (banda simetría):

$$\phi(0) + (1-c)\alpha\phi'(0) \leq \phi(\alpha) \leq \phi(0) + c\,\alpha\phi'(0)$$

con $c \in (0, \frac{1}{2})$.



Métodos de búsqueda lineal inexactos

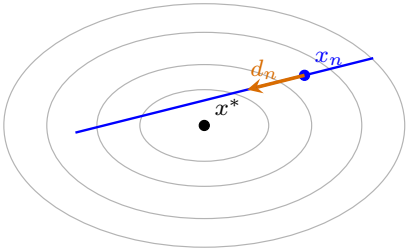
Se usan dentro de algoritmos de optimización Dado x_n y dirección de descenso d_n,

$$\phi(\alpha) = f(x_n + \alpha\, d_n),$$

Buscamos $\alpha_n > 0$ tal que cumpla alguna de las condiciones de Wolfe o Goldstein. Entonces el nuevo punto es

$$x_{n+1} = x_n + \alpha_n d_n$$

.



Métodos de búsqueda lineal inexactos

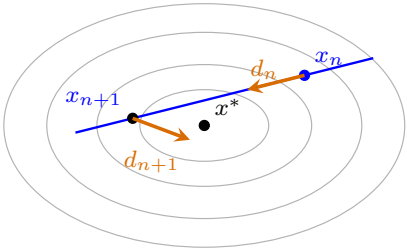
Se usan dentro de algoritmos de optimización Dado x_n y dirección de descenso d_n,

$$\phi(\alpha) = f(x_n + \alpha\, d_n),$$

Buscamos $\alpha_n > 0$ tal que cumpla alguna de las condiciones de Wolfe o Goldstein. Entonces el nuevo punto es

$$x_{n+1} = x_n + \alpha_n d_n$$

.



Métodos de primer orden en $\mathbb{R}^n$

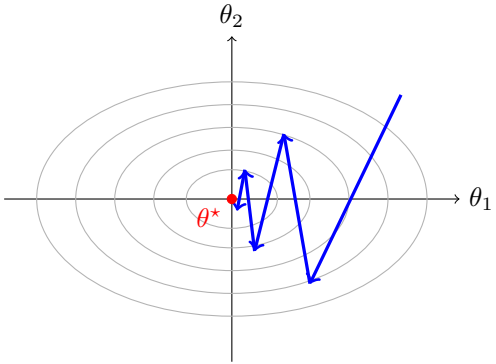
Idea: Localmente, la "mejor"dirección de descenso es

$$d_n = -\nabla f(x_n)$$

Entonces

$$x_{n+1} = x_n - \alpha_n \nabla f(x_n)$$

- GD paso constante: $\alpha_n = \alpha$
- GD paso variable prefijado: $\alpha_n = \frac{1}{n}$
- GD paso variable con búsqueda lineal inexacta
- GD paso variable con búsqueda lineal exacta



Condiciones de convergencia: métodos de primer orden

- **GD paso constante** $\alpha_n = \alpha$

  Si f es L-suave ($\|\nabla^2 f\| \leq L$) y convexa:

$$\alpha < \tfrac{2}{L} \;\Rightarrow\; f(x_n) - f^\star = \mathcal{O}\left(\tfrac{1}{n}\right)$$

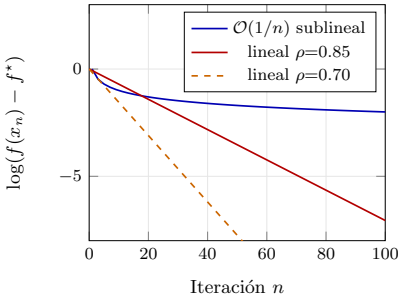
  Si además f es μ-fuertemente convexa:

$$\|x_n - x^\star\|^2 \leq \left(1 - \frac{\mu}{L}\right)^n \|x_0 - x^\star\|^2$$

  Tasa lineal: condicionamiento $\kappa = L/\mu \geq 1$ determina la velocidad.

$$\|x_n - x^\star\|^2 \leq \left(1 - \frac{\mu}{L}\right)^n \|x_0 - x^\star\|^2$$

$$f(x_n) - f^\star \leq (1 - \mu/L)^n (f(x_0) - f^\star)$$



Tasa sublineal $\mathcal{O}(1/n)$ (convexa):

$$f(x_n) - f^\star \leq \frac{C}{n}$$

Tasa lineal $\mathcal{O}(\rho^n)$ (fuertemente convexa):

$$f(x_n) - f^\star \leq C\,\rho^n \qquad \rho = 1 - \frac{\mu}{L} < 1$$

Condiciones de convergencia: métodos de primer orden

- **GD paso variable** $\alpha_n = 1/n$ Con f convexa y ∇f Lipschitz:

$$f(x_n) - f^\star = f(\bar{x}_n) - f^\star \leq \frac{C}{\sqrt{n}}, \quad \bar{x}_n = \tfrac{1}{n}\sum x_k$$

  Convergencia más lenta; útil cuando L es desconocida.

- **GD con búsqueda lineal inexacta** Si f es convexa y ∇f es Lipschitz:

$$f(x_n) - f^\star = \mathcal{O}\left(\tfrac{1}{n}\right)$$

  Con fuerte convexidad: tasa lineal independiente de la elección de α_n.

- **GD búsqueda exacta**
  $\alpha_n = \arg\min_\alpha f(x_n - \alpha \nabla f(x_n))$
  Con μ-fuerte convexidad y L-suavidad:

$$\|x_n - x^\star\|^2 \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^{2n} \|x_0 - x^\star\|^2$$

En todo caso ∇f Lipschitz. La fuerte convexidad aumenta la tasa de convergencia de $\mathcal{O}(1/n)$ a lineal.

Limitaciones prácticas de métodos clásicos en ML

- Bajo ciertas condiciones los métodos de primer y segundo orden son muy eficientes.
- En ML frecuentemente no se cumplen a lo que se suma la alta dimensión que hace impráctico calcular el gradiente (o el Hessiano) sobre todos los datos en cada iteración.
- En la práctica: se busca aproximar el gradiente utilizando subconjuntos de datos (**mini-batch**), lo que lleva a los métodos estocásticos y variantes modernas.
- Uso de algoritmos especialmente diseñados para grandes volúmenes de datos y entrenamiento distribuido.

GD — descenso en ML

Función objetivo:

$$F(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(y_i, f(x_i; \theta)\big)$$

Descenso por Gradiente (GD):

$$\theta_{k+1} = \theta_k - \eta \nabla F(\theta_k),$$

donde

$$\nabla F(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell\big(y_i, f(x_i; \theta)\big)$$

- Determinista: usa todo el dataset en cada paso.
- Simple de analizar, caro para n grande.

SGD y minibatch — Definición

Problemas GD en ML:

- Costo por iteración: $\mathcal{O}(n)$ gradientes.
- En la práctica, n y la latencia hacen inviable GD puro.

Idea: Aproximar gradiente por "minibatches."

$$\theta_{k+1} = \theta_k - \eta g_k, \quad g_k = \frac{1}{|B_k|} \sum_{(x_i, y_i) \in B_k} \nabla_\theta \ell\big(y_i, f(x_i; \theta_k)\big)$$

- B_k: minibatch aleatorio, con cardinalidad $|B_k| \ll n$.
- g_k es estimador no sesgado de $\nabla F(\theta_k)$.

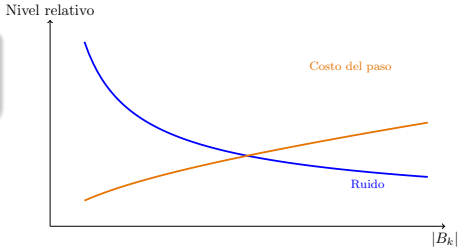
Ruido estocástico y descomposición

SGD = GD con ruido

$$g_k = \nabla F(\theta_k) + \xi_k,$$

Donde ξ_k es ruido,"$\mathbb{E}[\xi_k \mid \theta_k] = 0$

- Ese ruido actúa como regularización implícita.
- Minibatches chicos inducen más ruido en g_k.
- Ruido moderado puede ayudar a evitar mínimos agudos.
- Prefiere mínimos más planos que generalizan mejor.
- La elección de $|B_k|$ afecta convergencia y generalización.



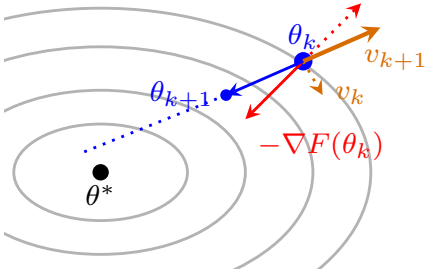
- Batch chico: pasos baratos, mayor varianza, posible inestabilidad.
- Batch grande: gradiente estable, mejor uso de hardware, menor ruido útil.

Momentum — Intuicion fisica

Momentum (heavy-ball)

$$v_{k+1} = \beta v_k + \nabla F(\theta_k), \quad \theta_{k+1} = \theta_k - \eta v_{k+1}$$

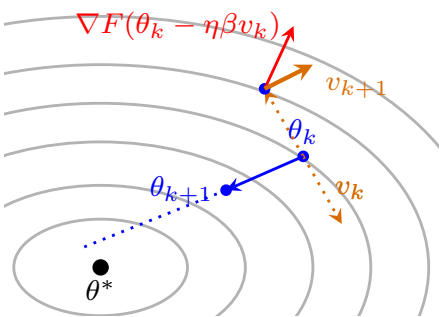
- $\beta \in [0, 1)$ controla memoria de gradientes pasados.
- Tipico en practica: $\beta \approx 0{,}9$.
- θ se mueve como una particula con inercia.
- El gradiente es fuerza; la velocidad acumula direccion.
- Reduce zigzag en valles angostos mal condicionados.



Nesterov — Mirada adelantada

$$v_{k+1} = \beta v_k + \nabla f(\theta_k - \eta\beta v_k), \quad \theta_{k+1} = \theta_k - \eta v_{k+1}$$

- Evalua gradiente en posicion anticipada.
- Suele corregir antes y evitar sobreoscilaciones.



Momentum vs Nesterov:

- Momentum: más simple, muy robusto en tuning inicial.
- Nesterov: respuesta más fina cerca del mínimo.
- En redes profundas, ambos compiten fuerte con AdamW.

Métodos adaptativos — Idea común

Problema con tasa global η:
Mismo η para todas las componente cuando son de distinto tamaño

Solución: paso efectivo *por coordenada*:

AdaGrad (Duchi et al., 2011)

Acumula todos los gradientes al cuadrado desde el inicio: $s_{k+1} = s_k + g_k \odot g_k, \quad s_0 = 0$

$$\theta_{k+1}^{(j)} = \theta_k^{(j)} - \frac{\eta}{\sqrt{s_{k+1}^{(j)}} + \epsilon} g_k^{(j)}$$

Pro: útil para datos esparsos (NLP, recomendación).
Contra: s_k solo crece $\Rightarrow$ paso $\to 0$ con el tiempo.

RMSProp (Hinton, 2012)

Reemplaza la suma por media exponencial con olvido ρ:

$$s_{k+1} = \rho\, s_k + (1-\rho)\, g_k \odot g_k$$

$$\theta_{k+1} = \theta_k - \eta \frac{g_k}{\sqrt{s_{k+1}} + \epsilon}$$

Pro: s_k no se satura; adapta a no estacionariedades.
Contra: sin corrección de sesgo inicial; sensible a ρ.

Otros métodos adaptativos - Adam

Adam (Kingma et al., 2014)

$$m_{k+1} = \beta_1 m_k + (1-\beta_1) g_k, \quad v_{k+1} = \beta_2 v_k + (1-\beta_2) g_k \odot g_k$$

$$\hat{m}_{k+1} = \frac{m_{k+1}}{1-\beta_1^{k+1}}, \quad \hat{v}_{k+1} = \frac{v_{k+1}}{1-\beta_2^{k+1}}, \quad \theta_{k+1} = \theta_k - \eta \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}$$

+ **Converge rápido al inicio:** los primeros pasos son grandes y en buena dirección.
+ **Poco tuning:** β_1=0,9, β_2=0,999, ϵ=10^{-8} funcionan bien por defecto en la mayoría de problemas.
+ **Estable:** la normalización por $\sqrt{\hat{v}}$ amortigua picos esporádicos.
+ **Robusto:** cada coordenada tiene su propio paso efectivo $\approx \eta\,\mathrm{sign}(g)$ cuando está bien calibrado.

– **Generaliza peor que SGD en visión:** en redes convolucionales entrenadas con muchas épocas, SGD+momentum con LR schedule fino suele alcanzar menor error de test. Adam converge a mínimos más agudos en estos casos.

– **Acople con weight decay:** en Adam clásico, $\lambda\|\theta\|^2$ se suma a la pérdida y el gradiente de regularización *también* se normaliza por $\sqrt{\hat{v}}$, lo que reduce su efecto real y hace el decay inconsistente.

Otros métodos adaptativos - AdamW

AdamW (Loschilov et al. 2017)

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon} - \eta \lambda \, \theta_k$$

El decay $\lambda \theta_k$ se aplica *directamente* sobre θ, no pasa por la normalización adaptativa. Controla la norma de θ de forma predecible y mejora la generalización. Opción por defecto en la mayoría de la implementaciones.

Calibración y validación

Cómo calibrar η en la práctica:

1. **LR Range Test:** barrer η de 10^{-7} a 10^{-1} y elegir el valor justo antes de que la pérdida de **validación** suba.
2. **Grid search corto:** probar $\eta \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ durante $\sim$10 % de las épocas; elegir el que minimiza la pérdida de **validación**.

Señales de η mal calibrado:

- Pérdida oscila sin bajar $\Rightarrow$ η demasiado grande.
- Pérdida baja muy lento $\Rightarrow$ η demasiado chico.
- Train baja, validación no $\Rightarrow$ overfitting, reducir η o regularizar.

Tres conjuntos, tres roles

Conjunto	Rol
Train	gradientes, update de θ
Validación	monitoreo, tuning de η
Test	evaluación final (una vez)

Importante

El algoritmo **sólo ve el train**. La validación orienta decisiones. El test **no se toca** hasta el final.

Early stopping

Detener cuando la métrica de **validación** no mejora tras una cantidad preestablecida de épocas (*patience*). Guardar el mejor θ observado (que no es nec. el último).

Ejemplo (GD vs Momentum vs Adam)

Consigna

Considere $f(\theta) = \frac{1}{2}(100\theta_1^2 + \theta_2^2)$, mal condicionada. Partiendo de $\theta_0 = (1, 1)$, escriba los updates de GD, Momentum y Adam para una iteración y explique por que aparece zigzag en GD.

Es una cuadrática como las de los notebooks de la U4, con número de condición $\kappa = 100$.

Entrega esperada

- Ecuaciones de actualización para cada método.
- Explicación geométrica del zigzag en el valle angosto.
- Cuál método esperaria más estable y por qué.

Solución guiada 1/3 — GD y zigzag

Para $f(\theta) = \frac{1}{2}(100\theta_1^2 + \theta_2^2)$:

$$\nabla f(\theta) = (100\theta_1, \theta_2)$$

GD:

$$\theta_{k+1} = \theta_k - \eta(100\theta_{1,k}, \theta_{2,k})$$

- Curvatura muy distinta por eje $\Rightarrow$ paso util en θ_2 puede ser excesivo en θ_1.
- Resultado: rebotes transversales (zigzag).

Solución guiada 2/3 — Efecto de Momentum

Momentum:

$$v_{k+1} = \beta v_k + \nabla f(\theta_k), \quad \theta_{k+1} = \theta_k - \eta v_{k+1}$$

- Promedia direcciones recientes y filtra oscilaciones de alta frecuencia.
- Reduce zigzag y acelera el avance en la direccion plana del valle.

Solución guiada 3/3 — Efecto de Adam

Adam (resumen):

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}$$

- Reescala por coordenada: eje con gradiente grande recibe paso efectivo menor.
- En cuadrática mal condicionada, amortigua fuerte el eje empinado θ_1.
- En práctica: menos zigzag inicial, pero validar generalización final.

Resumen

- GD se entiende con suavidad Lipschitz y tasas en convexos.
- SGD/minibatch introduce ruido: riesgo para estabilidad, beneficio para generalización.
- Momentum/Nesterov mitigan zigzag; AdamW es baseline adaptativo moderno.
- Tuning profesional requiere LR, schedule, warmup, clipping y early stopping.

Conexión con la materia

Todo lo de hoy son variantes del esquema de descenso de la U4: cambia cómo se estima el gradiente (SGD), cómo se combina con los pasos anteriores (momentum) o la métrica en la que se mide el paso (métodos adaptativos, como hace Newton). Para pérdidas no diferenciables, como Lasso o la *hinge* de la SVM, el análogo es el método del subgradiente de la U10.

Para leer

- Nocedal, Wright, *Numerical Optimization* (capítulos de métodos de primer orden).
- Kingma, Ba, *Adam: A Method for Stochastic Optimization*.
- Loshchilov, Hutter, *Decoupled Weight Decay Regularization (AdamW)*.
- Bottou, Curtis, Nocedal, *Optimization Methods for Large-Scale Machine Learning*.
- Ruder, S. *An Overview of Gradient Descent Optimization Algorithms*.